FilmDB 数据库性能实验

文件程序、PostgreSQL 与 openGauss

PostgreSQL 14.24 / openGauss 6.0.1 / C++17

姓名： 学号：

01 实验内容与方法

本实验用三种方式处理同一份电影数据：C++ 程序直接读写文件，或通过 PostgreSQL、openGauss 执行 SQL。比较的操作包括按标题搜索电影、按编号查找电影、批量修改姓名、查询人物参与的电影，以及按国家和年份统计片长。随后增加客户端数量，比较两个数据库每秒能完成多少次查询和更新。

PostgreSQL 在扫描、聚合、姓名更新和主键查询中都比 openGauss 快，并发吞吐量也更高。连接查询中，两个数据库都远快于逐行读取文件的程序。给文件建立索引后，单条主键查询又比数据库更快。

数据集

数据来自课程提供的 `filmdb.sql`，使用国家、电影、人物和演职关系四张表。除原始数据外，再生成 10 倍和 100 倍数据，比较数据量增加后的运行时间。

规模	电影	人物	演职关系
1×	9,204	16,489	46,850
10×	92,040	164,890	468,500
100×	920,400	1,648,900	4,685,000

国家表在三个规模下均为 185 行。100 倍数据的四个 TSV 文件合计约 161.1 MiB。

运行环境与测量方法

实验服务器运行 Ubuntu 22.04，配备双路 AMD EPYC 7542 和 NVMe 存储。基础实验为数据库分配 8 个 CPU 核、客户端分配 2 个核，两个数据库轮流运行。每个数据库的内存上限为 8 GiB；文件程序、客户端和运行脚本共用 4 GiB 内存上限。服务器还运行其他任务。

两库的 shared_buffers 均设为 512 MB，work_mem 为 4 MB，关闭单条查询内部的并行执行。更新完成后等待数据持久化。串行查询先预热一次；并发测试先预热 5 秒，再测量 20 秒。每组通常重复三次，表中给出中位数，图中的误差线表示最小值和最大值。

02 数据量增加后，哪些操作变慢

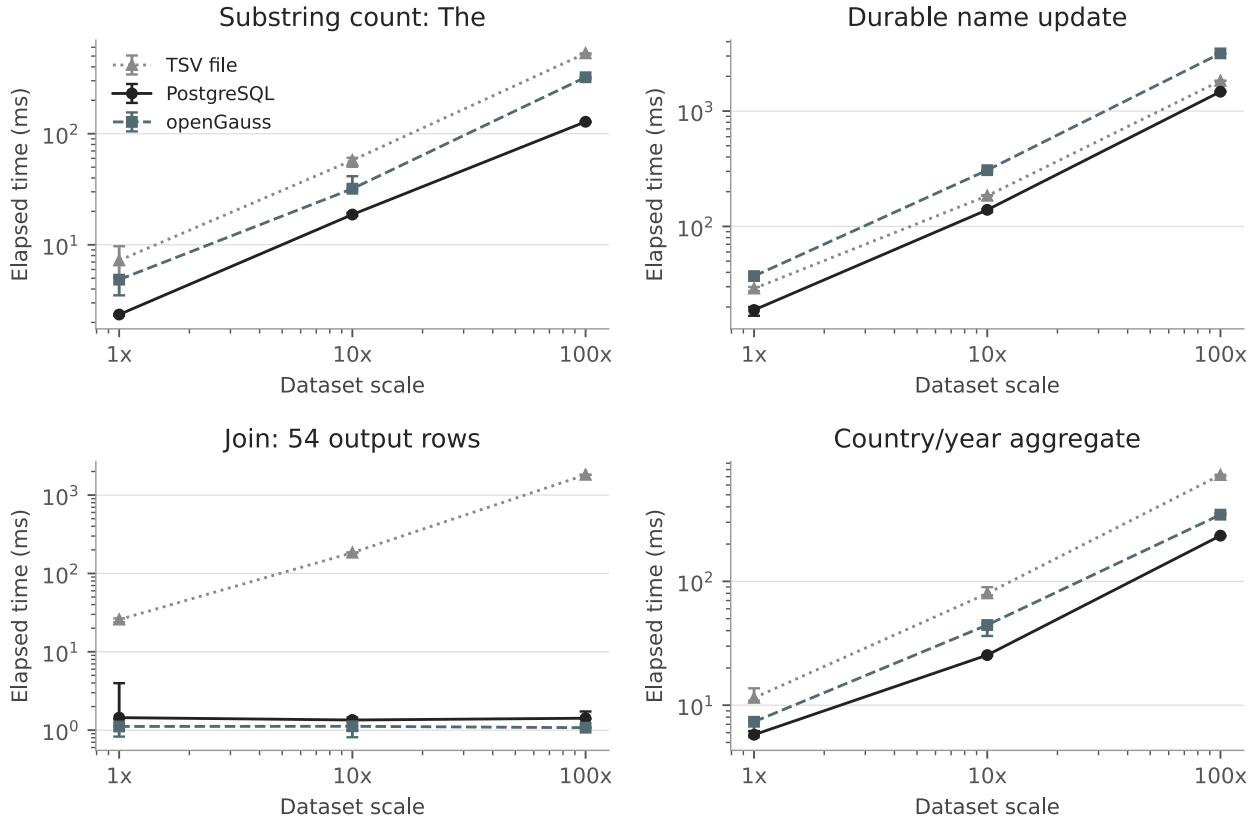


图 1　三个数据规模的耗时。横纵轴均使用对数刻度。

扫描和聚合需要读取整张表，耗时随数据量增加。100 倍数据下，PostgreSQL 的标题计数用时 128.13 ms，openGauss 为 322.35 ms；按国家和年份聚合分别用时 234.02 ms 和 345.67 ms。文件程序逐行解析 TSV，聚合时用 `std::map` 保存各组统计量；两库执行计划采用 HashAggregate。

100× 耗时 / ms	文件	PostgreSQL	openGauss
The 子串计数	529.23 [515.62, 529.85]	128.13 [126.06, 128.68]	322.35 [312.23, 324.21]
姓名同步更新	1827.72 [1819.72, 1836.73]	1472.21 [1471.99, 1481.63]	3162.42 [3132.07, 3169.66]
人物连接	1814.34 [1811.70, 1816.68]	1.42 [1.39, 1.73]	1.07 [1.06, 1.09]
国家 / 年份聚合	722.42 [703.17, 726.08]	234.02 [230.59, 234.67]	345.67 [345.26, 356.47]

表内为中位数，下方为［最小值，最大值］。

连接查询的差距最大。文件程序先扫描演职关系表，再扫描电影表；数据库通过人物索引找到关联记录，再按电影主键读取详情。三个规模都返回 54 行，因此数据库耗时变化很小，文件程序却随表的增大明显变慢。100 倍数据下，文件用时约 1.81 秒，两库都只需约 1 毫秒。

姓名更新把名字和姓氏中的 T 替换为 Ttt，100 倍数据共修改 128,700 行。文件程序写入临时文件并同步到磁盘后替换原文件；数据库在事务中执行 UPDATE 并提交。每轮开始前恢复原始姓名，避免在上轮结果上继续替换。

03 返回结果与主键查询

标题搜索分别使用 The、Love 和 Star，区分大小写。SQL 用 `strpos(title,$1)>0`，文件程序用 `string::find`。除了统计匹配数量，还测量返回所有匹配电影编号和标题的耗时。

100× 关键词	返回行数	文件 / ms	PG / ms	OG / ms
The	156,300	554.47	209.60	402.67
Love	8,800	528.57	125.68	313.70
Star	5,000	524.14	125.15	310.20

三个关键词都需要扫描电影表，但返回的记录数量相差很大。以 PostgreSQL 为例，The 匹配 156,300 行，只计数时用时 128.13 ms，返回编号和标题时增加到 209.60 ms。返回 Love 和 Star 的结果均用时约 125 ms。除了读取数据，程序还要传输和处理匹配记录，返回量较大的 The 查询因此更慢。

给文件建立索引后，主键查询很快

100×，100 次点查	单次查询 / µs	准备 + 100 次查询 / s
文件扫描	262765.74	26.828
文件偏移索引	4.84	0.741
PostgreSQL	97.43	0.569
openGauss	194.43	0.588

顺序查找需要从文件开头逐行寻找电影编号，平均一次约 263 ms。文件索引保存每个编号在文件中的位置，查询时先查内存中的哈希表，再跳到相应位置读取记录，平均只需 4.84 µs。PostgreSQL 和 openGauss 的主键查询分别为 97.43 µs 和 194.43 µs。

文件索引适合这类按编号读取一条记录的简单操作，查找和读取都在同一个进程中完成。数据库查询还需要客户端与服务端通信，并经过 SQL 执行流程。另一方面，文件索引也需要事先建立并装入内存。

准备时间如何计算

表中最后一列包括生成查询编号、装载索引或建立数据库连接，以及执行 100 次查询的时间。实验程序在准备阶段会扫描电影文件以生成随机查询编号，因此数据库的准备时间也包含文件读取。文件索引虽然单次查询最快，但这一小批查询加上准备时间后共用 0.741 秒，PostgreSQL 为 0.569 秒。

100 倍数据的文件索引另需 0.946 秒建立，未计入表中最后一列。索引和数据库连接都可以反复使用，查询次数越多，平均到每次查询上的准备成本越小。

04 约束与事务

演职关系中的电影编号应当对应一部存在的电影。数据库用外键检查这条规则：向演职关系表插入一个不存在的电影编号时，PostgreSQL 和 openGauss 都拒绝了该操作，并返回外键错误 23503。

接着测试一笔包含两步操作的事务。第一步修改一部电影的片长，第二步插入非法演职关系。第二步失败后执行 ROLLBACK，再次读取片长，检查第一步的修改是否一起撤销。

实验	PostgreSQL	openGauss
插入不存在电影的演职关系	拒绝；23503	拒绝；23503
先更新 runtime，再触发外键错误	中间值已变化	中间值已变化
回滚后检查片长和四张表	全部恢复	全部恢复

两库在事务中都能读到修改后的片长，回滚后都恢复了原值。程序还比较了四张表的行数和内容摘要，结果与事务开始前一致。

文件程序也能通过写临时文件再替换原文件来完成更新，但它没有外键检查和多步事务。如果应用需要同时修改几张表，还要自行处理修改顺序、失败恢复以及表之间的一致性。数据库把这些工作集中在约束和事务中，应用只需定义规则并提交操作。

检查不同查询条件

连接查询选择四个人物编号，分别返回 0、1、22 和 54 行，检查空结果和不同数量的关联记录。文件程序与两个数据库的返回内容一致。

查询条件	人物编号	返回行数
无匹配	164900	0
少量关联	1	1
中等关联	1438	22
较多关联	151852	54

主键查询采用两种编号分布：均匀随机选择，以及把 90% 的请求集中在前 1% 的电影编号上。每轮执行 10,000 次，重复 3 轮，三个实现的返回结果均一致。

这组检查运行时服务器较忙，因此只核对查询结果，耗时未纳入性能比较。

05　客户端如何影响并发吞吐量

并发测试使用 10 倍数据，每个客户端收到结果后再发送下一条请求。先测试 1、4、8 个客户端：PostgreSQL 的吞吐量较高，但增加客户端后增长逐渐放缓；openGauss 的增长则更接近直线。为找到 PostgreSQL 增长放缓的原因，继续测试 16 和 32 个客户端，并记录两端 CPU 用量。

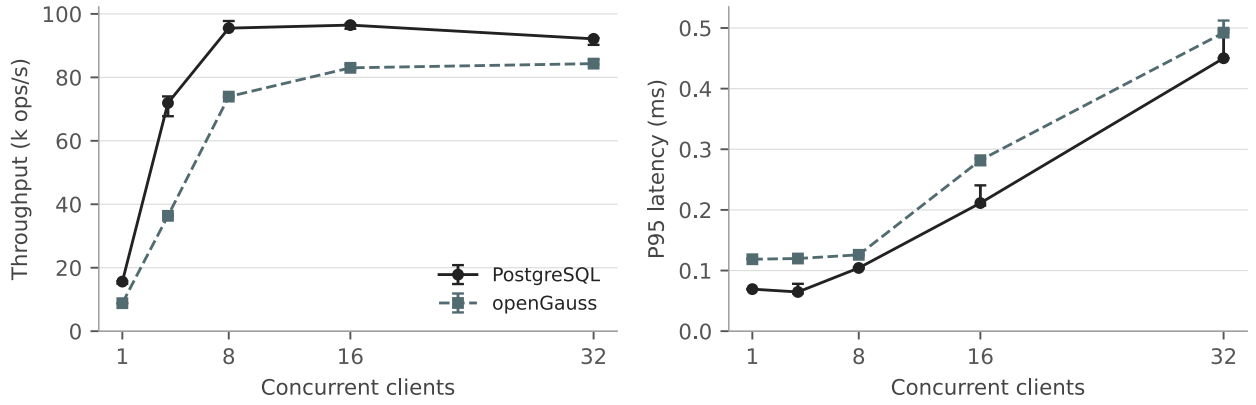


图 2　数据库分配 8 核，客户端分配 2 核。P95 表示约 95% 的请求能在这一时间内完成。

32 客户端纯读	客户端用量 / 核	数据库用量 / 核
PostgreSQL	2.00	4.06
openGauss	1.88	7.99

PostgreSQL 的客户端已经用满分配到的 2 个核，而数据库只用了约 4 个核。客户端要生成请求、收取结果、计算摘要并记录延迟，这些工作先占满了 CPU。openGauss 则相反：数据库已接近 8 核满载，客户端还有少量余量。

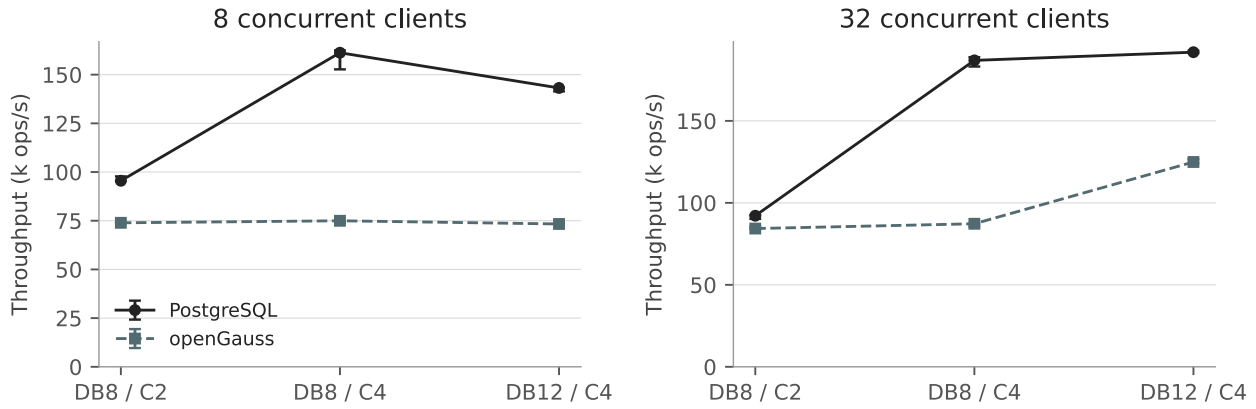


图 3　增加 CPU 后的吞吐量。DB 表示数据库核数，C 表示客户端核数。

保持数据库 8 核，把客户端从 2 核增加到 4 核，PostgreSQL 的吞吐量明显上升，openGauss 变化较小。随后保持客户端 4 核，把数据库增加到 12 核，openGauss 的吞吐量也随之上升。

06 结果分析与结论

数据库核 / 客户端核	PG / 万次•s⁻¹	OG / 万次•s⁻¹
8 / 2	9.21	8.43
8 / 4	18.69	8.72
12 / 4	19.19	12.49

32 客户端纯读；三轮吞吐中位数。

32 个客户端下，给客户端增加两个核后，PostgreSQL 从每秒 9.21 万次提高到 18.69 万次，约为原来的两倍。数据库核数没有变化，说明它此前仍有处理能力，只是客户端来不及发送和接收更多请求。openGauss 在数据库从 8 核增加到 12 核后，从每秒 8.72 万次提高到 12.49 万次，主要瓶颈在数据库一侧。

进一步给数据库和客户端各分配 8 核，PostgreSQL 达到每秒 21.82 万次，数据库此时接近 8 核满载；openGauss 为每秒 8.51 万次。将关键配置的测量时间延长到 120 秒后，增加客户端核数使 PG 吞吐量变为原来的 2.03 倍，增加数据库核数使 OG 变为 1.42 倍。

读写混合负载

混合负载中，约 90% 的请求按主键读取电影，10% 更新一部电影的片长，每条更新单独提交。基础实验的结果如下。

8 客户端	PG / 万次•s⁻¹	OG / 万次•s⁻¹
只读	9.41	7.35
90% 读 + 10% 同步写	9.43	5.50

加入更新后，PostgreSQL 的吞吐量基本不变，openGauss 从每秒 7.35 万次降到 5.50 万次。对于这组以读取为主、夹杂少量单行更新的请求，PostgreSQL 处理得更快。

如何选择实现方式

如果只需按编号读取静态文件，文件偏移索引简单且快。需要关联多张表、修改数据并维护一致性时，数据库更方便：索引避免反复扫描，外键拦截非法数据，事务负责把一组修改一起提交或撤销。

在两个数据库之间，FilmDB 实验更支持选择 PostgreSQL。它在标题搜索、分组聚合、批量姓名更新、主键查询和读写混合负载中都优于 openGauss。人物连接查询由 openGauss 小幅领先，100 倍数据下两者分别为 1.07 ms 和 1.42 ms。

本实验的数据可以放入内存，查询经过预热，每轮更新前重新导入数据。如果继续研究，下一步应增加超过内存的数据，并持续执行更新，观察磁盘读取和旧版本清理对耗时的影响。

07 实现与复现

主程序为 `filmbench.cpp`，使用 Linux C++17、libpq 和 pthread。`--backend` 选择文件、PostgreSQL 或 openGauss，`--op` 选择查询、更新、导入或并发测试。建表 SQL 已内置，完整项目包含数据准备、实验运行和报告生成脚本。

项目地址：github.com/luanmenglei/dbh_project1

编译与运行

```
make
python3 scripts/prepare.py --scales 1
./build/filmbench --backend file --op search_count \
  --data data/generated/s1 --keyword The
./build/filmbench --backend file --op build_index \
  --data data/generated/s1
```

数据库连接参数从环境变量 `FILMBENCH_DSN` 读取。配置连接后，以下命令重建实验 schema、导入数据，再检查外键和回滚。

```
./build/filmbench --backend postgres --op load \
  --data data/generated/s1 --schema filmbench_s1 --reset
./build/filmbench --backend postgres --op functional \
  --schema filmbench_s1
```

openGauss 客户端使用其配套 libpq 编译。依赖安装、实例部署和完整实验命令见 README。

数据准备和计时

`prepare.py` 从课程 SQL 中提取四张表并生成 TSV。扩容时复制电影、人物和演职关系，给各副本的编号增加偏移量，使关联关系保持对应；`replica` 字段区分不同副本中标题、姓名相同的记录。国家表保持不变。

查询计时包括执行、返回结果和计算结果摘要。当前程序分别记录请求准备、后端准备和查询时间；第 03 节使用原实验的准备总时间。索引建立单独计时，每个规模测一次。两库开启 fsync、同步提交和 full_page_writes；并发延迟用桶宽约 1% 的直方图统计，CPU 时间每 0.5 秒采样一次。

执行计划与实验记录

100 倍人物连接的单次 EXPLAIN 记录中，PG 的 Execution Time 为 0.480 ms，OG 的 Total runtime 为 0.735 ms。EXPLAIN 在另一次连接中执行，记录服务端诊断时间；正文表格使用预热后、包括结果接收的三轮计时中位数。

结果目录（均在 results/ 下）	内容
epyc-compact-evidence	三个规模的查询、更新与并发测试
scaling-probe / scaling-balance scaling-confirmation	调整 CPU 核数及 120 秒测量
review-correctness review-coverage-diverse	独立参考程序、外键、回滚与查询结果检查

各目录保存原始输出、配置和源码快照。scripts/build_typst_report.py 核对记录后生成图表、编译 PDF，并更新提交文件的 SHA-256 清单。